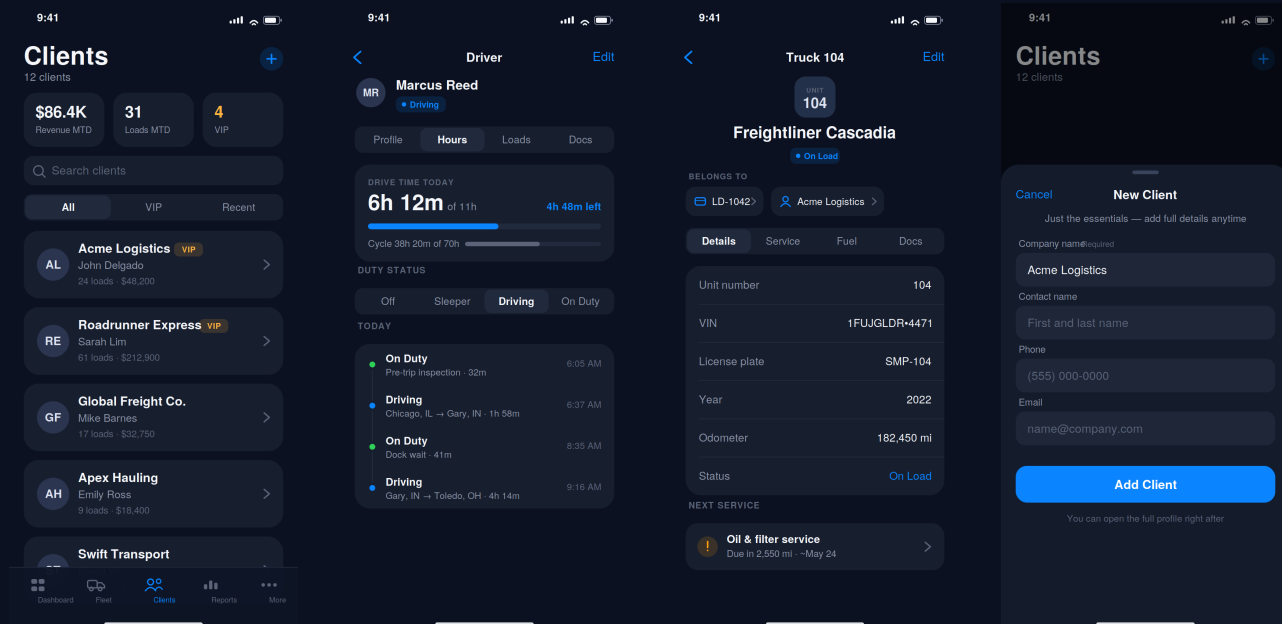


DriveCommand

Mobile Design System and Page Build Guide

The single source of truth for how every screen in the DriveCommand mobile app looks, feels, and behaves. Built on Apple's design language: depth instead of borders, restraint instead of decoration, and one accent color used sparingly.



v1.2 · July 2026 · 13 reference screens · Owner portal: Clients, Trucks, Drivers

Reference language: iOS Health, Wallet, Things 3 (dark mode)

How to build with this guide

This document does the thinking; you do the running. Every page has a mockup, a prompt for Claude Code, and a checklist. Follow the loop below exactly and the app comes out right. If a built screen and its mockup disagree, the mockup wins.

- 1 Put this guide in the repo (once)**
Copy this PDF and the mockups folder into the repository — a docs or planning folder at the repo root works; just keep the PDF and the PNGs together and remember the path.
- 2 Run the Foundation prompt (once)**
Open the repo in VS Code, open Claude Code, paste Prompt 0 from Section 10. It builds the design tokens, the shared component kit, and .planning/mobile-design-system.md. Commit when it finishes. Do not start any page before this is done.
- 3 One page at a time, in order**
Pages go Clients, then Trucks, then Drivers; within each entity Overview, Quick Create, Detail. Copy that page's prompt from Section 10, paste into Claude Code, let it finish completely.
- 4 Compare against the mockup**
Run the app on a real device and hold it next to the page's PNG. Anything different — spacing, corner radius, text size, color, missing state — tell Claude Code exactly what differs and let it fix. Repeat until they match.
- 5 Tick every box**
Open the page's checklist in this PDF and the Global QA list (Section 09). Physically check every box. A page with one unchecked box is not done — it is not 'close enough'.
- 6 Commit, then next page**
One commit per page, named after the page. Never work on two pages at once.
- 7 Feedback goes to the system first**
If a rule needs to change, it changes in .planning/mobile-design-system.md and this PDF first, then in every affected screen. Rules never change on a single page.

The one-sentence brief

When a user opens any screen, it should feel like Apple built it: soft, quiet, instantly understandable, and impressive before they have read a single word.

Design principles

Depth, not borders

Cards never have outlines or borders. They separate from the background purely through surface contrast (#171E2E on #0B1120). If you are typing `borderWidth` on a card, stop.

Status is a whisper

States use translucent tints (color at 14–15% opacity) with the color only in a small dot and the label text. No filled chips, no colored card borders — including VIP.

Continuous 20px corners

Cards 20, inputs 12, sheets 24 on top. Everything soft. Nothing square.

Real data rhythm

Rows never repeat identical numbers. Empty states invite action in one line. Loading states are content-shaped skeletons, never spinners.

One accent, spent carefully

#0A84FF appears only on: the active tab, primary buttons, tappable text actions, and live/positive emphasis. Never on containers, never as decoration.

Native iOS controls only

Large-title headers, pill search fields, segmented controls, bottom sheets with grabbers, frosted tab bars. No FAB anywhere — add actions live top-right as a tinted circle, or as the primary button inside sheets.

People get circles, machines get squares

Human entities (drivers, contacts) use circular monogram avatars. Vehicles use rounded-square unit chips. This is how users tell rows apart at a glance.

View and edit are the same page

Same fields, same order, same sections in both modes. View reads label-left / value-right; edit stacks the label above a full-width input so long values never truncate while typing.

Design tokens

Hard-code nothing. Every color, size, and space below becomes a named token in the codebase (NativeWind theme). If a value is not in these tables, it does not belong in the app.

Color

	bg	#0B1120	Screen background
	card	#171E2E	All cards and list rows — never with a border
	elevated	#212A3D	Segmented thumb, pressed states
	input	#1E2638	Text field fill (edit mode and sheets)
	hairline	#232C40	Dividers inside grouped rows only
	text.primary	#F5F7FA	Titles, values
	text.secondary	#8B93A7	Labels, sublines
	text.muted	#5B6478	Meta lines, placeholders, inactive icons
	accent	#0A84FF	The only brand color in the UI
	vip	#FFB340	VIP tag only, at 15% opacity fill
	success	#30D158	Ready / on-duty / paid
	warning	#FF9F0A	Expiring docs, service due, in shop
	danger	#FF453A	Destructive actions, overdue, HOS violation
	avatar	#2B3550	Monogram and unit-chip fill

Typography (SF Pro on device — weights and sizes are the contract)

Large title	34 / Bold	Overview page titles
Page title	22 / Bold	Detail header entity name
Nav title	17 / Semibold	Centered stack-header titles
Row title	17 / Semibold	Entity name in list rows
Body	15–16 / Regular	Values, inputs, sublines
Caption	13 / Regular	Meta lines, helper text
Section label	12 / Semibold, uppercase, +0.8 tracking	Group headers like RECENT LOADS
Tag	11 / Bold	VIP and status pill labels
KPI value	22 / Bold	Numbers inside KPI cards

Spacing & radius

Spacing scale (only these 5): 8, 12, 16, 20, 24	Screen margin 20 · Card gap 12	Card padding 16 · Section spacing 24
Card radius 20 (continuous)	Input radius 12	Sheet top radius 24
Row height: list 92–96, field 52	Avatar 44 (list) / 72 (detail)	Touch target min 48

Component rules

Card

Fill card, radius 20, no border, no shadow needed on dark. Content padding 16–18. Whole card is tappable with pressed state opacity 0.75 and light haptic.

Monogram avatar

Circle, avatar fill, two initials in 600 weight #DDE3F0. No photos until the user uploads one.

Status pill

Translucent tint at 14%: 3px dot + 11.5 Semibold label in the status color. Height 22, radius 8.

KPI card

Value 22 Bold (tinted only when it maps to a status), label 12 secondary below. 2–3 across, height 74, radius 18. Tapping a KPI applies that filter to the list below.

Search field

Pill radius 13, card fill, magnifier + placeholder in muted. Appears on every Overview.

Sheet input

Label 13 secondary above field; 46-high input fill radius 12; 'Required' hint in muted after the label — required is marked by words, not asterisks.

Field group — view mode

One card, 52-high rows: label left secondary, value right primary Medium. Hairline dividers inset 18, none under the last row. Status values get their tint.

Parent strip (Belongs to)

On detail pages with parent records: a BELONGS TO section label + tappable chips (40 high, radius 13, card fill, accent entity icon, name, chevron) directly under the identity header and above the tabs. Tapping navigates up to the parent record. Parents live here only — never as field rows or inline sections.

Empty state

Centered icon in muted, one 15pt line ('No clients yet'), one 13pt secondary line inviting the add action. No illustrations.

Entity list row

Left: avatar/chip 44. Middle: title 17 Semibold, subtitle 14–15 secondary, meta 13 muted. Right: status pill top-right OR chevron centered — pill pages drop the chevron (the whole card is the tap target).

Unit chip (vehicles)

44 rounded-square radius 12, avatar fill, tiny UNIT eyebrow over the number.

VIP tag

11pt Bold amber text on amber at 15% opacity, radius 6.5, sits right of the name. Never a border, never a filled badge.

Segmented control

Native iOS style: track = card fill, selected thumb = elevated fill, radius 11/9. Used for list filters and detail tabs.

Bottom sheet (Quick Create)

Dimmed 45% backdrop, sheet fill #141B2B, top radius 24, 36x5 grabber, Cancel top-left, centered 17 Semibold title, one-line purpose subtitle.

Primary button

Full width, 50 high, radius 15, accent fill, 17 Semibold white. Disabled = 35% opacity. Exactly one per sheet.

Field group — edit mode

Same card, same field order and sections. Each field: 13pt secondary label above a full-width 44-high input (input fill, radius 10). No hairlines — the inputs separate the rows. Long values never truncate.

Tab bar

Frosted (blur) over content, hairline top edge, 5 items, active = accent icon + label, inactive = muted. Labels always visible, 10pt.

Add action

Top-right of Overview: 32 circle, accent at 16% fill, accent plus glyph. Opens Quick Create. No floating action button anywhere in the app.

Skeleton loader

Content-shaped blocks in elevated fill, shimmer 800ms. Never a spinner on data screens.

The four-page pattern

Every entity (Clients, Trucks, Drivers — and later Loads, Trips, Invoices) ships exactly four pages. Once a user learns one entity, they know the whole app.

1 · Overview

The list. Large title + count, add button top-right, KPI cards (tap = filter), search, segmented filter, entity rows. Answers 'how is this part of my business doing?' in one glance.

2 · Quick Create

A bottom sheet with only the required fields (3–4 max) and one button. For the owner standing at a truck or on the phone with a broker. Everything else is added later on the Detail page.

3 · Detail — View

The full record. Identity header (avatar/chip, name, status), contact actions where the entity is a person or company, stat strip, segmented tabs for child records (loads, service, hours, docs), then grouped field rows.

4 · Detail — Edit

THE SAME PAGE. Nav swaps to Cancel / Save, contact actions hide, and the field group re-renders in edit mode: identical fields, order, and sections, each label stacked above a full-width input. The user already knows where every field lives — that is the whole point.

View / Edit parity rules

- ☐ One screen component with an `isEditing` flag — not two screens. Layout code is shared by construction.
- ☐ Field order, grouping, and vertical position are identical in both modes.
- ☐ Edit fields stack the label above a full-width input (input fill, radius 10) — field order and section grouping identical to view mode.
- ☐ Save is disabled until something changed; Cancel with unsaved changes asks to discard (native alert).
- ☐ Success: save, light success haptic, return to view mode in place — no navigation, no full-screen reload.
- ☐ Hierarchy: parents in the Belongs-to strip above the tabs (navigate up); children as tabs (navigate down). No third pattern.

Form & field intelligence

The schema has more fields than any mockup shows, and it will keep growing. These rules tell Claude Code how to take ANY set of fields and organize them so users are never overwhelmed, always know what to enter next, and feel progress. Every build prompt in Section 10 points here.

Discover, never assume

Before building any page, read the Prisma schema for the entity, its relations, and the mobile API response types. List every available field. The mockups show representative fields — the schema is the truth.

Classify every field

Sort each field into: Identity (names, numbers that identify), Status (computed states — display only, never editable), Contact, Operational (the day-to-day working data), Financial, Compliance/Legal, and Metadata (createdAt, ids — usually hidden). Unsure? Ask: 'when does the owner need this?'

Three exposure tiers

List row: max 3 lines — one title, one subtitle, one meta line with at most 2 stats. Quick Create: only the fields required to make the record real, 3–5 hard max. Detail page: everything, grouped. A field missing from a row or sheet is not lost — it lives on Detail.

Group in cards of 3–7

Detail fields render as grouped cards with section labels, never one endless list. Order groups by how often the owner reaches for them: Identity+Contact first, Operational, Financial, Compliance, Metadata last (or omitted). A group over 7 fields splits or hides its long tail behind 'Show more'.

Order fields like a conversation

Within a group: the field a human says first goes first (name before number, company before contact). Pairs stay adjacent (city/state, license/expiry). A field that depends on another comes after it. Required fields come before optional ones in create flows.

Make progress visible

Quick Create is one screen, one button — completing it must feel like an achievement, with the toast offering the next step ('View' the new record). Multi-section entry shows position ('2 of 4'). Detail pages may show a quiet completeness hint ('Profile 70% complete · Add license') that gives the owner a next action, never a nag.

Never overwhelm

Sensible defaults for everything that has one (status, dates, units, next unit number). Empty optional fields display as an em dash in view mode — calm, not demanding. Inputs appear only in edit mode or sheets. If in doubt, show less and make it one tap to see more.

Right input, right words

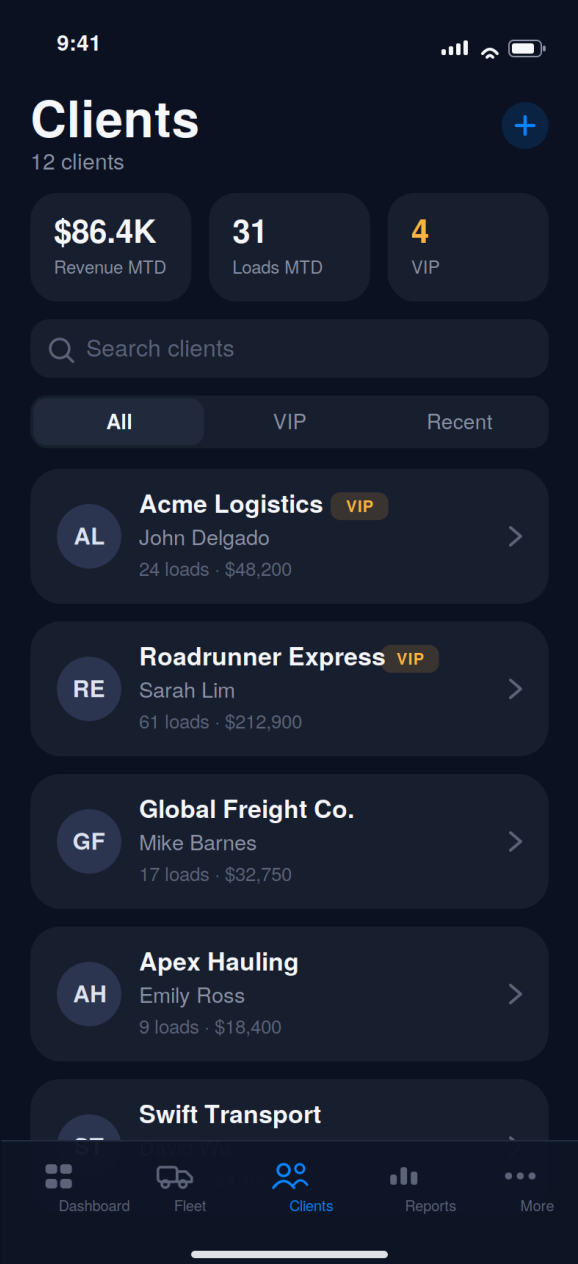
Keyboard matches the field (phone pad, email, numeric). Enums and dates get native pickers, never free text. Money and distances format live. Labels use the owner's words, not column names — 'DBA name', never 'dba_name'. Validate on submit only; errors inline under the field; scroll to the first error; input is never lost.

Clients

The CRM. Data: company name, contact name, phone, email, VIP priority, load count, total revenue, recent loads, activity, notes. The owner opens this page to answer one of two questions: 'who do I call?' or 'who is worth my time?'

Everything on these four pages serves those two questions.

Clients — Overview



The user's situation

The owner wants a read on their book of business in two seconds, then to reach one specific client. They are scanning, not reading — names must pop, numbers must be honest.

Layout, top to bottom

- Large title 'Clients' + live count subtitle; add button top-right opens Quick Create.
- KPI cards: Revenue MTD, Loads MTD, VIP count (amber value). Tap VIP = applies the VIP filter.
- Search filters company AND contact name as you type.
- Segmented: All / VIP / Recent (recent = last interaction desc).
- Rows: monogram, company 17 Semibold (+VIP tag), contact name, 'N loads · \$revenue' meta, chevron.
- Sorted by total revenue desc by default.

Developer checklist

- ☐ No borders on any card; VIP shows only as the small translucent tag.
- ☐ Load counts and revenue vary per row and come from real aggregates (totalLoads, totalRevenue).
- ☐ Whole row is tappable (min 48pt) with pressed opacity + light haptic, navigating to Detail.
- ☐ List is FlashList with pull-to-refresh and content-shaped skeletons while loading.
- ☐ Empty state: 'No clients yet' + one line pointing at the add button.
- ☐ Search and filters combine (VIP + query works together).
- ☐ Tab bar shows Clients active in accent; large title collapses to nav title on scroll.

Clients — Quick Create

9:41

Clients

12 clients

+

Cancel New Client

Just the essentials — add full details anytime

Company name^{Required}

Acme Logistics

Contact name

First and last name

Phone

(555) 000-0000

Email

name@company.com

Add Client

You can open the full profile right after

The user's situation

The owner is on the phone with a new broker and has 20 seconds. They need the client to exist NOW; paperwork comes later. Anything beyond four fields is friction that loses the moment.

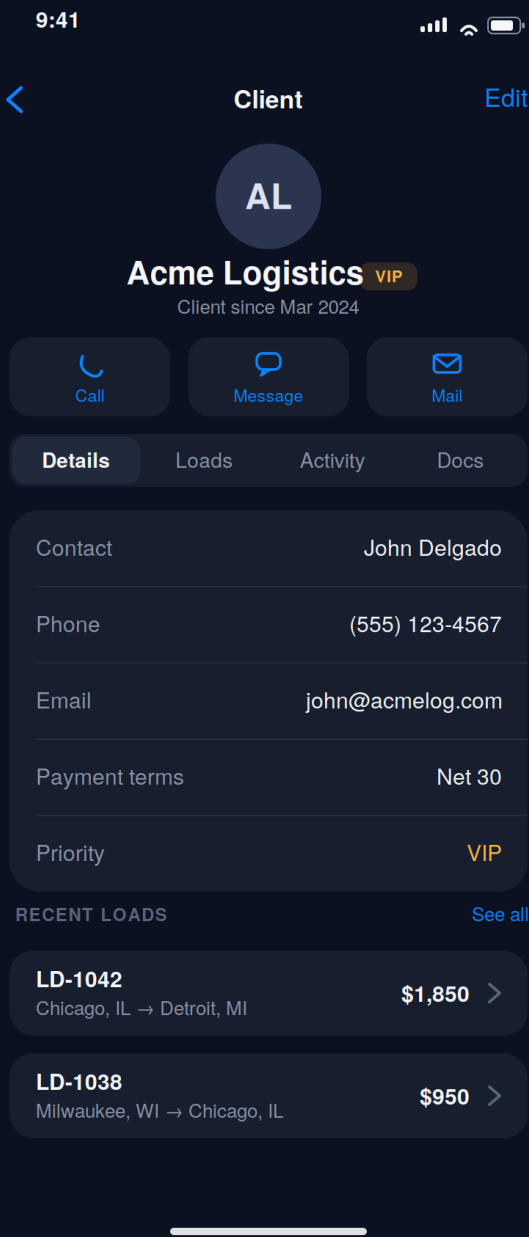
Layout, top to bottom

- Bottom sheet over the dimmed Overview; grabber, Cancel, centered 'New Client'.
- Subtitle sets expectations: 'Just the essentials — add full details anytime'.
- Fields: Company name (Required), Contact name, Phone, Email — in that order.
- One primary button 'Add Client'; caption under it promises the full profile is one tap away.

Developer checklist

- ☐ Company name is the only required field; button disabled (35% opacity) until it has a value.
- ☐ Keyboard types match fields (phone pad, email keyboard); return key advances to next field.
- ☐ Required marked with the word 'Required' in muted — no asterisks.
- ☐ Validation on submit only; inline error text under the field, field keeps its value.
- ☐ On success: success haptic, sheet dismisses, new client visible in list, toast offers 'View' to open Detail.
- ☐ Sheet dismisses by grabber swipe or Cancel; unsaved input asks to discard.
- ☐ API failure: toast + sheet stays open with values intact.

Clients — Detail (View)



The user's situation

The owner tapped a client to act: call them, check what they're worth, or review recent loads. Contact actions must be reachable with a thumb without scrolling.

Layout, top to bottom

- Nav: back chevron, 'Client', Edit top-right.
- Identity: 72 monogram, company 22 Bold + VIP tag, 'Client since' caption.
- Contact actions row: Call / Message / Mail cards — native deep links (tel:, sms:, mailto:).
- Segmented tabs: Details / Loads / Activity / Docs.
- Field group: Contact, Phone, Email, Payment terms, Priority (VIP value tinted amber).
- RECENT LOADS section: load number, route, rate, chevron to load detail; 'See all' jumps to Loads tab.

Developer checklist

- ☐ Edit (top-right) flips this exact page into edit mode — no navigation.
- ☐ Contact actions open the native dialer/messages/mail with values prefilled.
- ☐ Tabs switch content below the header without pushing new screens.
- ☐ Field rows: label left secondary, value right primary; hairlines inset 18, none under last row.
- ☐ Missing values render as '—' in muted, never blank rows removed (parity with edit).
- ☐ Loads tab reuses the standard load row component from the Loads section.
- ☐ Header identity block collapses on scroll so tabs pin below the nav bar.

Clients — Detail (Edit)

9:41

Cancel

Edit Client

Save

AL

Change Photo

Acme Logistics VIP

Details

Loads

Activity

Docs

Contact

John Delgado

Phone

(555) 123-4567

Email

john@acmelog.com

Payment terms

Net 30

Priority

VIP

NOTES

Prefers morning pickups. Dock 4 only.

Same fields, same order — labels stack above full-width inputs

The user's situation

The owner spotted a wrong phone number in view mode. Their eyes are already on that row — edit mode must keep the row exactly where it was, now typeable. Zero re-orientation.

Layout, top to bottom

- Nav swaps to Cancel / 'Edit Client' / Save (Save in accent Semibold).
- Avatar gains 'Change Photo'; contact action row hides (its data is now editable below).
- SAME field group, same order, same sections — each label stacks above a full-width input (input fill, radius 10).
- Priority row becomes a selector (Standard / VIP); Notes becomes a multiline input.
- Tabs remain visible but non-editing tabs are disabled while editing.

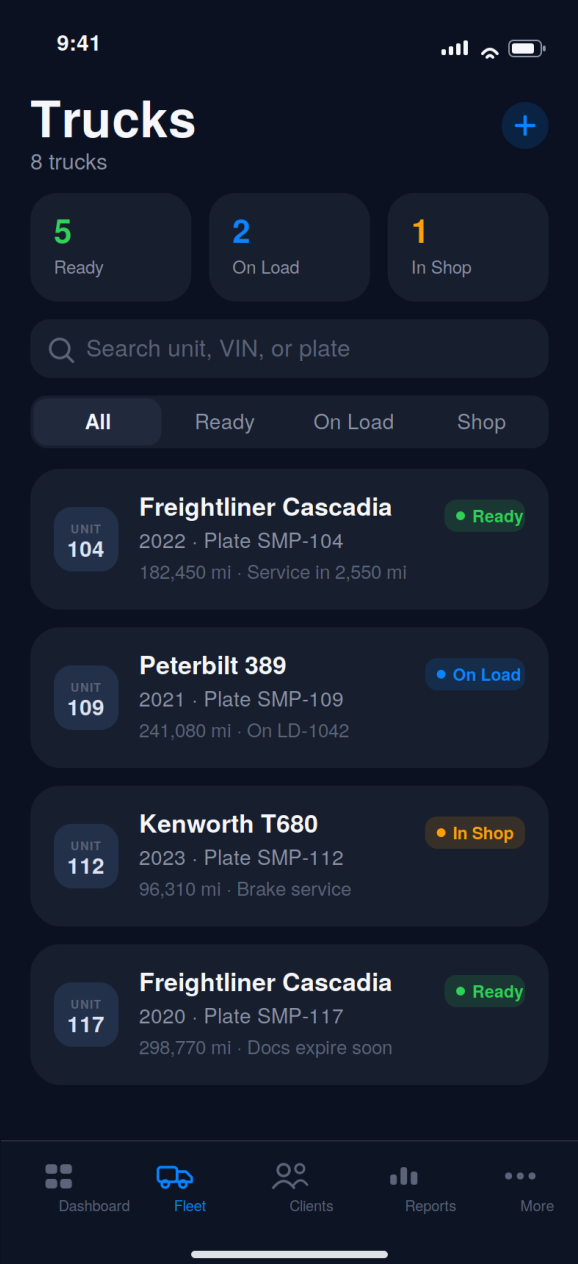
Developer checklist

- ☐ Implemented as the SAME component as Page 3 with an `isEditing` state — verify by diffing: only nav, inputs, photo action, and notes differ.
- ☐ Save disabled until a field actually changes.
- ☐ Cancel with changes: native 'Discard Changes?' alert.
- ☐ Save: success haptic, return to view mode in place, updated values visible immediately.
- ☐ Validation errors appear inline under the offending row; scroll snaps to the first error.
- ☐ Keyboard never covers the focused input (keyboard-avoiding scroll).

Trucks

The fleet. Data: unit number, VIN, plate, make, model, year, odometer, computed status (Ready / On Load / In Shop / Expired Docs), scheduled service, fuel records, documents. The owner's question here is always operational: 'what can roll today, and what is about to cost me money?'

Trucks — Overview



The user's situation

The owner checks this before dispatching: which trucks are free, which are stuck. Status must be readable from across the room; the meta line warns about money (service due, docs expiring).

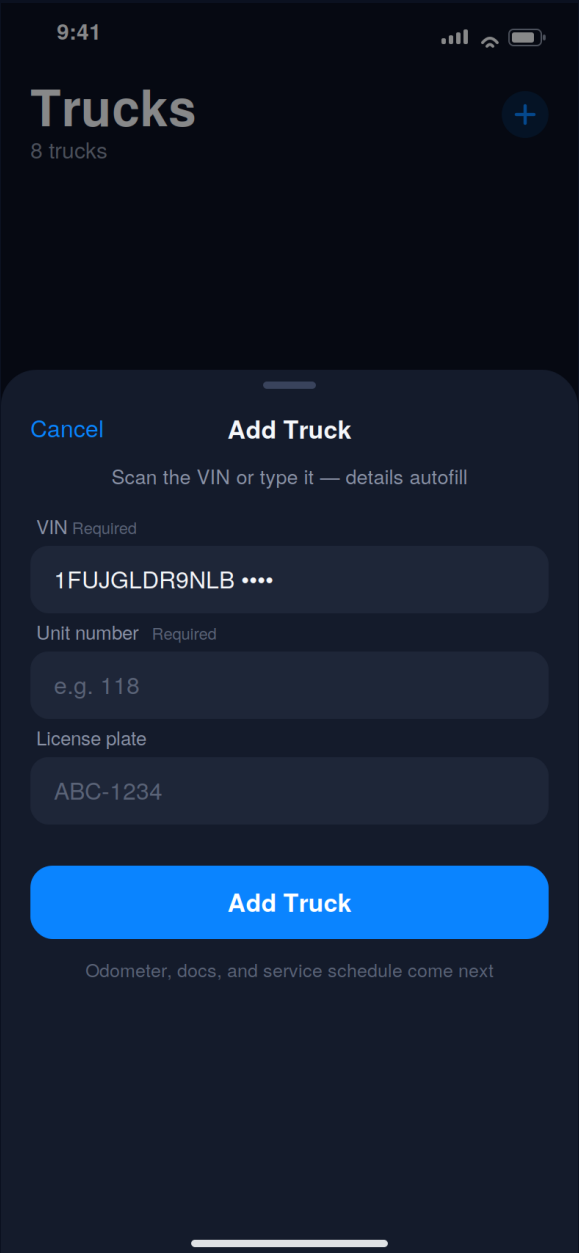
Layout, top to bottom

- Large title + count, add button top-right.
- KPI cards mirror the status system: Ready (green), On Load (accent), In Shop (orange). Tap = filter.
- Search hits unit, VIN, and plate.
- Segmented: All / Ready / On Load / Shop.
- Rows: rounded-square UNIT chip, make+model 17 Semibold, year · plate subtitle, odometer · alert meta, status pill top-right (no chevron on pill rows — the whole card navigates).

Developer checklist

- ☐ Status is computed server-side (computeTruckStatus) — never stored or guessed on the client.
- ☐ Unit chip, not a circle — vehicles are not people.
- ☐ Meta line surfaces the most urgent fact per truck: next service, active load, or doc expiry.
- ☐ KPI counts always equal the sum of list rows under the matching filter.
- ☐ Pill colors map to tokens exactly: Ready=success, On Load=accent, In Shop=warning, Expired=danger.
- ☐ Skeletons, pull-to-refresh, empty state per the global rules.

Trucks — Quick Create



The user's situation

The owner is standing in front of a newly purchased truck. VIN first — it can be scanned or typed off the door jamb, and it autofills make/model/year so they type almost nothing.

Layout, top to bottom

- Sheet: 'Add Truck', subtitle 'Scan the VIN or type it — details autofill'.
- Fields: VIN (Required, with camera-scan icon in the field), Unit number (Required), License plate.
- Primary button 'Add Truck'; caption: odometer, docs, and service schedule come next (on Detail).

Developer checklist

- ☐ VIN decodes on blur (NHTSA vPIC or existing decoder) and shows resolved make/model/year as a confirmation line under the field.
- ☐ VIN field validates 17 chars, uppercases input, rejects I/O/Q.
- ☐ Unit number suggests next free number (e.g. 118) as placeholder.
- ☐ Duplicate VIN in tenant = inline error, not a server 500.
- ☐ Success: haptic, dismiss, truck appears with Ready status, toast offers 'View'.
- ☐ Same sheet mechanics as Clients Quick Create — reuse the sheet component, do not rebuild it.

Trucks — Detail (View)

9:41

<

Truck 104

Edit

UNIT

104

Freightliner Cascadia

On Load

BELONGS TO

LD-1042 >

Acme Logistics >

Details

Service

Fuel

Docs

Unit number

104

VIN

1FUJGLDR•4471

License plate

SMP-104

Year

2022

Odometer

182,450 mi

Status

On Load

NEXT SERVICE

Oil & filter service

Due in 2,550 mi · ~May 24

>

The user's situation

The owner opens a truck to answer: is it healthy, is it legal, what has it cost lately. The next service card is the single most valuable block — it prevents breakdowns.

Layout, top to bottom

- Nav: back, 'Truck 104', Edit.
- Identity: UNIT chip, 'Freightliner Cascadia' 22 Bold, status pill centered.
- BELONGS TO strip: when on an active load, tappable chips for the Load (LD-1042) and its Client navigate up to those records.
- Segmented tabs: Details / Service / Fuel / Docs.
- Field group: Unit, VIN (truncated middle, tap to copy), Plate, Year, Odometer, Status (tinted).
- NEXT SERVICE card: warning-tinted icon, service name, 'Due in N mi · ~date', chevron to Service tab.

Developer checklist

- ☐ Same detail skeleton as Clients — header, tabs, field group. Reuse components.
- ☐ VIN row: tap copies full VIN with a 'Copied' toast.
- ☐ Odometer shows thousands separators; updating it lives in Edit mode.
- ☐ Service tab lists ScheduledService items with overdue/due-soon/ok states in token colors.
- ☐ Docs tab: document rows with expiry dates; expired = danger tint, <30 days = warning tint.
- ☐ Status pill matches the Overview computation exactly (same endpoint, same logic).

Trucks — Detail (Edit)

9:41

Cancel

Edit Truck

Save

UNIT

104

Freightliner Cascadia

Status is computed — not editable

Unit number

104

VIN

1FUJGLDR73LK•4471

License plate

SMP-104

Year

2022

Odometer

182,450

Same fields, same order — labels stack above full-width inputs

The user's situation

Odometer corrections and plate changes happen monthly. The owner should fix a number in five seconds flat because every field is exactly where view mode showed it.

Layout, top to bottom

- Nav: Cancel / 'Edit Truck' / Save.
- Identity header condenses (chip + name + VIN caption), pill hides while editing.
- SAME field group in edit mode — labels stacked above full-width inputs, identical field order; Status is NOT editable (computed) and is stated in the header caption.
- Next-service card hidden in edit (managed from the Service tab).

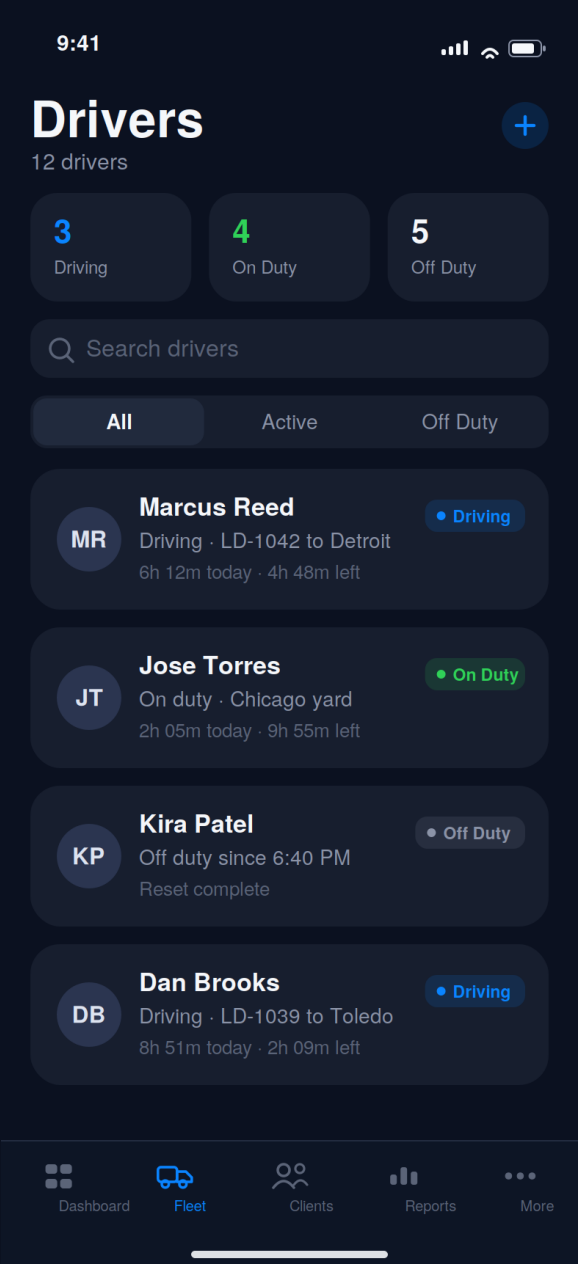
Developer checklist

- ☐ Same component as Page 3 with isEditing — verify by diff.
- ☐ Odometer input: numeric pad, strips commas on input, reformats on blur; rejects values lower than current unless confirmed.
- ☐ VIN editable only until the truck has loads/fuel/docs attached; after that, locked with explanation.
- ☐ Save disabled until dirty; Cancel-with-changes alert; success haptic and in-place return.
- ☐ Validation inline under rows; first error scrolled into view.

Drivers

The people. Data: name, phone, email, license number and expiry, DOB, address, hire date, duty status, hours of service (HOS), assigned loads, compliance documents. Two jobs: keep the owner legal (hours, licenses, docs) and keep dispatch moving (who is available right now).

Drivers — Overview



The user's situation

Dispatch view. The owner needs 'who can take this load' — which is duty status plus remaining hours. The meta line does the HOS math so the owner never has to.

Layout, top to bottom

- Large title + count, add button (opens Invite Driver).
- KPI cards: Driving (accent), On Duty (green), Off Duty (neutral). Tap = filter.
- Search by name.
- Segmented: All / Active / Off Duty.
- Rows: monogram, name 17 Semibold, current activity subline ('Driving · LD-1042 to Detroit'), hours meta ('6h 12m today · 4h 48m left'), duty pill top-right.

Developer checklist

- ☐ Duty pill colors: Driving=accent, On Duty=success, Off Duty=neutral secondary.
- ☐ Subline links the live load context, not generic text — pull from active load assignment.
- ☐ Hours meta comes from HOS entries, updates on refresh, and turns warning when <2h remain.
- ☐ Compliance issues (expired license/doc) add a small warning dot on the avatar.
- ☐ Row tap opens Detail on the Profile tab; tapping the hours meta area opens the Hours tab directly.
- ☐ Skeletons, pull-to-refresh, empty state per global rules.

Drivers — Quick Create

9:41

Drivers

12 drivers

Invite Driver

They finish onboarding on their own phone

First name Required

Marcus

Last name Required

Reed

Phone Required

(555) 000-0000

Send Invite

License, docs, and pay setup happen in onboarding

The user's situation

Hiring is a handshake moment — the owner adds the driver from the yard in 15 seconds and the driver's own phone does the heavy lifting. This is DriveCommand's professional-onboarding differentiator: the sheet only starts the relay.

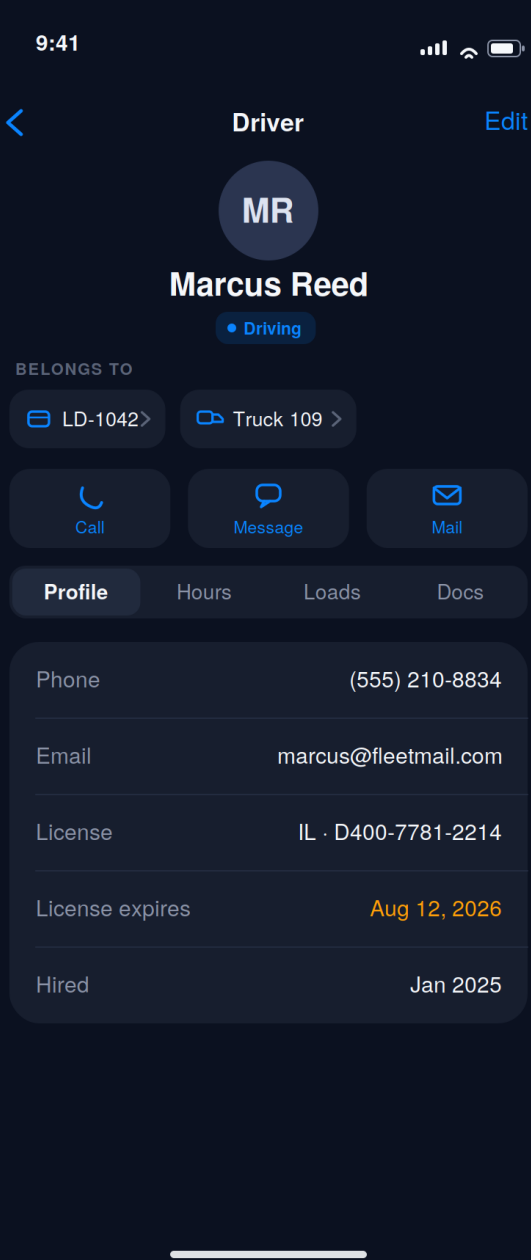
Layout, top to bottom

- Sheet: 'Invite Driver', subtitle 'They finish onboarding on their own phone'.
- Fields: First name (Required), Last name (Required), Phone (Required).
- Primary button 'Send Invite'; caption: license, docs, and pay setup happen in onboarding.

Developer checklist

- ☐ Sends the existing DriverInvitation flow (SMS/email invite, PENDING to ACCEPTED).
- ☐ Phone validates as usable mobile format before enabling the button.
- ☐ Success: haptic, dismiss, driver appears in list with an 'Invited' neutral pill until they accept.
- ☐ Resending an invite is available from the driver's Detail page, not by creating twice — duplicate phone shows inline error pointing at the existing driver.
- ☐ Reuse the shared sheet + input components.

Drivers — Detail (View + Edit)



The user's situation

The owner opens a driver to call them, verify a license, or check availability. Personal data is sensitive — the page should feel calm and factual, like an iOS contact card. (Edit mode, shown on the next page's pattern, follows the exact Clients parity rules.)

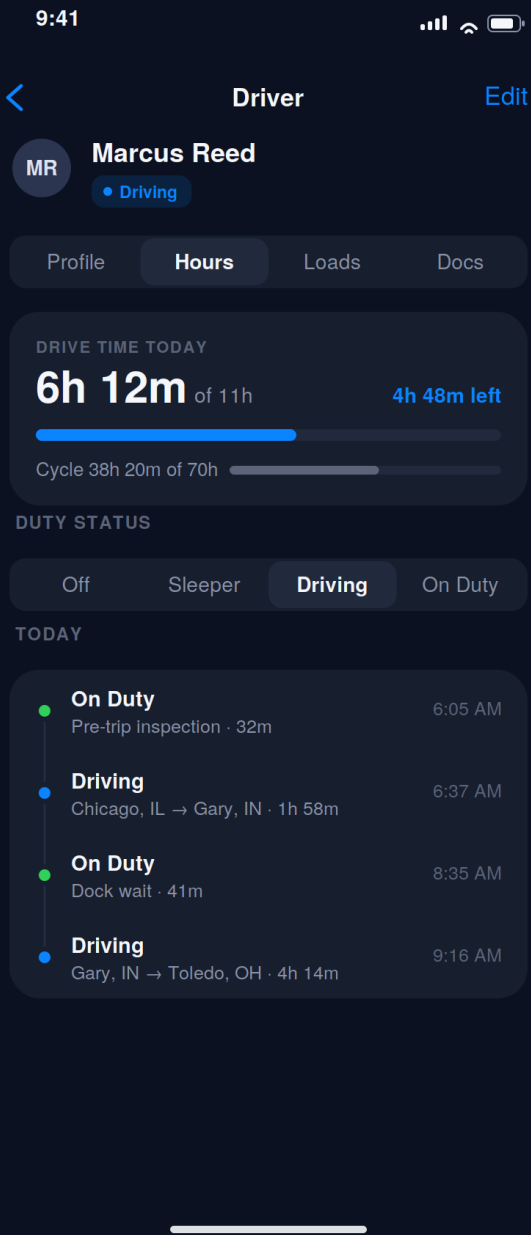
Layout, top to bottom

- Nav: back, 'Driver', Edit.
- Identity: 72 monogram, name 22 Bold, duty pill.
- BELONGS TO strip: chips for the active Load and assigned Truck navigate up.
- Contact actions: Call / Message / Mail.
- Segmented tabs: Profile / Hours / Loads / Docs.
- Field group: Phone, Email, License, License expires (warning tint when <90 days), DOB, Hired.

Developer checklist

- ☐ License expiry drives color: >90d normal, <90d warning, past = danger + banner on Docs tab.
- ☐ Edit mode: same page, isEditing flag, same parity checklist as Clients Page 4 — including keyboard avoidance and inline validation.
- ☐ Docs tab reuses the document row component from Trucks (one component, both entities).
- ☐ Loads tab lists assigned loads with the standard load row.
- ☐ Address is a single row that expands; tapping it in view mode opens Maps.

Drivers — Hours (HOS tab)



The user's situation

This tab keeps the owner legal. One glance answers: can this driver keep driving, and for how long. Numbers are big, math is done, violations are impossible to miss.

Layout, top to bottom

- Condensed header: small avatar, name, duty pill — Hours tab active in the segmented tabs.
- DRIVE TIME TODAY card: 30pt elapsed vs 11h limit, accent 'remaining' figure, progress bar; cycle line beneath (38h 20m of 70h) with its own thin bar.
- Duty status selector: Off / Sleeper / Driving / On Duty as a segmented control (owner view is read-mostly; driver app writes it).
- TODAY timeline: dot-and-line entries — status, description with location/duration, timestamp.

Developer checklist

- ☐ Timeline dots use duty colors (Driving=accent, On Duty=success, Off/Sleeper=neutral).
- ☐ Progress bar turns warning at 80% of the limit, danger at 100%; a violation adds a danger banner.
- ☐ Times render in the tenant's timezone; durations as '4h 14m', never decimals.
- ☐ Day picker (chevrons or swipe) to review previous days; today is the default.
- ☐ Data source: DriverHOSEntry records (OFF_DUTY, SLEEPER_BERTH, DRIVING, ON_DUTY).
- ☐ Empty day: 'No duty entries' + last known status, never a blank card.

Global QA checklist — run on every page

After a page's own checklist passes, run this list. These are the rules users feel without being able to name them — this is where 'smooth' lives.

Look

- ☐ Zero borders/outlines on cards anywhere on the page.
- ☐ Only #0A84FF as accent; count its uses — if it appears more than ~4 times, something is decorated.
- ☐ Radii: cards 20, inputs 12, sheets 24. No square corners visible.
- ☐ Text uses only the 9 type styles from Section 02 — no one-off sizes.
- ☐ Every color on screen exists in the token table.

States

- ☐ Loading = content-shaped skeleton (800ms shimmer). No spinners.
- ☐ Empty = one inviting line + a path to the add action.
- ☐ Error = toast + the user's input preserved. Never a dead end.
- ☐ Offline = cached data shown with a quiet 'Updated Xm ago' line, not a blocking error.

Feel

- ☐ Every tappable element $\geq 48\text{pt}$ with pressed state (opacity 0.75) and the mapped haptic.
- ☐ Screen mounts with a subtle fade; sheets spring; tab switches are instant (no animation).
- ☐ Scrolling is 60fps with 50+ rows (FlashList, memoized rows).
- ☐ Keyboard never covers a focused input; return key advances or submits.
- ☐ Pull-to-refresh works on every list with a light haptic on trigger.

Consistency

- ☐ This page shares components with its siblings — sheet, inputs, field group, rows, pills are imports, not copies.
- ☐ View and Edit are one component (diff the file to prove it).
- ☐ Wording matches the app vocabulary: Add Client / Add Truck / Send Invite — the button verb equals the toast verb.
- ☐ Accessibility: every icon-only control has an accessibilityLabel; text respects Dynamic Type one step up.

SECTION 10 · COPY-PASTE THESE INTO CLAUDE CODE

Build prompts

One prompt per page, in build order. Copy the whole gray block, paste it into Claude Code in VS Code, press enter, and let it finish. Every prompt tells Claude Code to discover the real fields from the schema itself — the mockups show the pattern, the codebase supplies the fields. When it finishes, do the mockup comparison and the checklists before moving on.

Prompt 0 — Foundation (run once, before everything)

After it runs: compare against nothing yet — commit when TypeScript passes.

Read the design guide PDF and mockup PNGs stored in this repo (search for DriveCommand-Mobile-Design-System) before writing any code. We are redesigning the mobile app (apps/mobile, React Native + Expo + NativeWind v4) to Apple's design language: iOS Health / Wallet / Things 3 in dark mode. This prompt builds the foundation only. Do not modify any existing screen.

1. Create .planning/mobile-design-system.md encoding: COLORS bg #0B1120, card #171E2E, elevated #212A3D, input #1E2638, hairline #232C40, txt #F5F7FA, txt2 #8B93A7, txt3 #5B6478, accent #0A84FF, vip #FFB340, success #30D158, warning #FF9F0A, danger #FF453A, avatar #2B3550. SPACING scale is exactly {8,12,16,20,24}: screen margin 20, card gap 12, card padding 16, section spacing 24. RADII: card 20, input 12, sheet top 24. TYPE: large title 34 bold, page title 22 bold, nav/row title 17 semibold, body 15-16, caption 13, section label 12 semibold uppercase, tag 11 bold, KPI 22 bold. RULES: no borders/outlines on cards ever; accent only on active tab, primary buttons, tappable text, live emphasis; no FAB (add = top-right tinted circle); status = translucent pill (14% fill, dot, label); people = circular monogram avatars, vehicles = rounded-square unit chips; skeletons never spinners; 48pt min targets. FORMS: view mode = label-left/value-right rows 52 high; edit mode = same fields, same order, same sections, label stacked above a full-width input. HIERARCHY: parents = tappable BELONGS TO chips above the tabs; children = segmented tabs. Also transcribe the Form & Field Intelligence rules from the guide.

2. Add the color tokens to tailwind.config.js with those semantic names.

3. Build a typed shared kit in apps/mobile/components/ui/ using NativeWind only (no StyleSheet, no hex outside the config): Screen, LargeTitleHeader, NavHeader, SearchField, SegmentedControl, KPICard/KPIRow, StatusPill, VIPTag, MonogramAvatar, UnitChip, EntityRow, FieldGroup (isEditing flag implementing the form rule), ParentStrip, SheetContainer, SheetInput, PrimaryButton (disabled = 35% opacity), SectionHeader, ContactActionsRow, EmptyState, Skeleton (800ms shimmer). Pressables: active opacity 0.75, light haptic, accessibilityLabel.

4. Grep all mobile screens and append a violation list (borders, hex literals, spinners, FABs, off-scale spacing) to the design doc as a migration checklist.

Confirm: npx tsc --noEmit passes; zero StyleSheet.create and zero hex literals in the new components.

Prompt 1 — Clients: Overview

After it runs: compare against clients-overview.png + the Clients Overview checklist (Section 06).

Read .planning/mobile-design-system.md and docs/mockup/clients-overview.png. Rebuild the mobile Clients list screen to match the mockup pattern exactly, using only the components in apps/mobile/components/ui/.

FIELD DISCOVERY (do this first): read prisma/schema.prisma for the client/customer model, its relations, and cached aggregates, plus the mobile clients API response. List every field, classify per the Form & Field Intelligence rules, then choose: one title field, one subtitle field, one meta line with max 2 stats, one priority/status marker. State what you left off the row and why.

BUILD: Screen with LargeTitleHeader (entity name, live count, AddButton opening Quick Create), KPIRow of 2-3 KPIs from the most decision-relevant aggregates you discovered (tapping a KPI applies its filter), SearchField matching the two most identifying fields, SegmentedControl with the 2-3 most useful filters, FlashList of EntityRow (MonogramAvatar initials, title, subtitle, meta, VIPTag if a priority field exists, chevron), sorted by the value field that best answers 'who is worth my time'. Pull-to-refresh, content-shaped Skeleton, EmptyState pointing at the add button. Search and filters must compose.

Confirm: tsc passes; zero borders, hex literals, FABs, or off-scale spacing in the diff; KPI counts equal filtered row counts.

Prompt 2 — Clients: Quick Create

After it runs: compare against [clients-quick-create.png](#) + the [Clients Quick Create checklist](#) (Section 06).

Read `.planning/mobile-design-system.md` and mockup `clients-quick-create.png`. Build the New Client bottom sheet, launched from the Clients Overview `AddButton`.

FIELD DISCOVERY: from the schema and create-API validation, determine the minimum field set that makes a client record real and useful – 3 to 5 fields hard max, required fields first, per the Form & Field Intelligence rules. Everything else is deferred to the Detail page; say so in the sheet's subtitle and closing caption so the user knows nothing is lost.

BUILD: `SheetContainer` (grabber, Cancel, centered title, one-line purpose subtitle) + `SheetInputs` with correct keyboard types, 'Required' spelled out (no asterisks), return key advancing. One `PrimaryButton`, disabled until required fields are valid. Validation on submit only; inline errors under fields; values always preserved, including on API failure (toast + sheet stays open). Success: light success haptic, dismiss, list refresh, toast with a View action opening the new record's Detail page – that toast is the progress moment.

Confirm: tsc passes; sheet mechanics identical to the shared `SheetContainer` (no local fork); duplicate-record errors surface inline, never as a raw server error.

Prompt 3 — Clients: Detail (view + edit, one component)

After it runs: compare against [clients-detail-view.png](#) and [clients-detail-edit.png](#) + both [Detail checklists](#) (Section 06).

Read `.planning/mobile-design-system.md` and mockups `clients-detail-view.png` / `clients-detail-edit.png`. Build the Client Detail page as ONE component with an `isEditing` flag – never two screens.

FIELD DISCOVERY: list every client field and relation from the schema and detail API. Classify into groups per the Form & Field Intelligence rules (Identity+Contact, Operational, Financial, Compliance, Metadata) and render them as grouped `FieldGroup` cards of 3-7 rows with `SectionHeaders`, ordered by how often the owner reaches for them. Empty optionals show an em dash. Identify child record types (loads, interactions, documents, and anything else you find) – each becomes a segmented tab. If the client has parent records, they go in a `ParentStrip` above the tabs.

VIEW: `NavHeader` (back, entity, Edit), identity header (72 `MonogramAvatar`, name 22 bold, priority tag), `ContactActionsRow` with native `tel:/sms:/mailto:` deep links, tabs, field groups, and a recent-child section with See all jumping to that tab. EDIT: nav swaps to Cancel / Save (accent), contact actions hide, the SAME field groups re-render per the edit form rule (label stacked above full-width input, identical order and sections); enums become native pickers; Save disabled until dirty; Cancel-with-changes shows a native Discard alert; save = success haptic and in-place return to view; inline validation, scroll to first error; keyboard-avoiding scroll.

Confirm: tsc passes; diff proves a single component; deep links open the native apps; tabs switch in place with no navigation pushes.

Prompt 4 — Trucks: Overview

After it runs: compare against trucks-overview.png + the Trucks Overview checklist (Section 07).

Read .planning/mobile-design-system.md and mockup trucks-overview.png. Rebuild the Trucks list screen.

FIELD DISCOVERY: read the truck model, its computed status logic (find the existing server-side status computation – do not recompute on the client), maintenance schedule, document expiries, and active load linkage. Choose the row content per the exposure-tier rule: title (make+model or unit), subtitle (year/plate class of fields), and a meta line that surfaces the SINGLE most urgent fact per truck (next service due, active load, or expiring doc) – write the priority logic that picks it.

BUILD: LargeTitleHeader + AddButton, KPIRow mirroring the status system with token tints (tap = filter), SearchField across the identifying fields you found (unit, VIN, plate class), SegmentedControl of the status filters, FlashList rows using UnitChip (vehicles are not people), StatusPill top-right, whole card navigates (no chevron on pill rows). KPI counts must equal filtered row counts. Skeletons, pull-to-refresh, EmptyState per the global rules.

Confirm: tsc passes; zero client-side status computation in the diff; pill tints map exactly to the token table.

Prompt 5 — Trucks: Quick Create

After it runs: compare against trucks-quick-create.png + the Trucks Quick Create checklist (Section 07).

Read .planning/mobile-design-system.md and mockup trucks-quick-create.png. Build the Add Truck sheet.

FIELD DISCOVERY: determine the minimum creating field set from schema + create validation. The owner is physically standing at the truck – order fields so the first one is the one on the truck itself (VIN-first if the schema supports decode). If a VIN decoder exists in the codebase use it; otherwise stub decodeVin() against the free NHTSA vPIC API. Decoded make/model/year render as a confirmation line, not extra inputs. Suggest smart defaults for anything derivable (next free unit number as placeholder).

BUILD: shared SheetContainer + SheetInputs; VIN validates 17 chars, uppercases, rejects I/O/Q; duplicate identifiers within the tenant produce inline errors naming the existing record. PrimaryButton disabled until valid. Success: haptic, dismiss, record appears with its computed status, toast with View. The closing caption tells the user what comes next on the Detail page (odometer, docs, service schedule) so a 3-field sheet still feels complete.

Confirm: tsc passes; the sheet reuses shared components; decode failures degrade gracefully to manual entry.

Prompt 6 — Trucks: Detail (view + edit)

After it runs: compare against trucks-detail-view.png and trucks-detail-edit.png + both checklists (Section 07).

Read .planning/mobile-design-system.md and mockups trucks-detail-view.png / trucks-detail-edit.png. Build Truck Detail as one component with isEditing, mirroring the Client Detail structure.

FIELD DISCOVERY: list all truck fields and relations. Groups per the intelligence rules; computed fields (status) are display-only – in edit mode they are omitted or shown locked with a one-line reason. Child record types you find (service/maintenance, fuel, documents, and anything else) become tabs. Parents: if the truck has an active load, render ParentStrip with tappable chips to the Load and its Client – this is the first ParentStrip use, make sure it generalizes.

VIEW: NavHeader, UnitChip identity header, StatusPill, ParentStrip, tabs, grouped fields (long identifiers middle-truncate with tap-to-copy + Copied toast; distances/odometer format with separators), and one 'costs me money soon' card (next service or nearest expiry) with a warning tint linking into its tab. Expiry-aware rows across tabs: expired = danger, near = warning, using the thresholds already in the codebase or 30 days. EDIT: stacked-label form rule, identical order; numeric fields get numeric pads, strip formatting on input, reformat on blur; guard odometer decreases behind a confirm; lock immutable identifiers once child records exist, with an explanation line. Standard save/cancel/validation behavior from the Client Detail prompt.

Confirm: tsc passes; single component; ParentStrip navigates; document row component is shared for reuse by Drivers.

Prompt 7 — Drivers: Overview

After it runs: compare against [drivers-overview.png](#) + the Drivers Overview checklist (Section 08).

Read `.planning/mobile-design-system.md` and mockup `drivers-overview.png`. Rebuild the Drivers list screen. This is a dispatch view: the owner is answering 'who can take this load'.

FIELD DISCOVERY: read the driver/user model, duty-status and hours-of-service data, active load assignment, license and document expiries. Build the row so the subtitle is live context from the active assignment (never generic text) and the meta line is the HOS math already done (elapsed today + remaining, warning tint when low). If remaining-time math is not available from the API, add it server-side or in a tested shared util – never inline in JSX. Compliance problems (expired license/doc) render as a small warning dot on the avatar.

BUILD: `LargeTitleHeader` + `AddButton` (opens Invite), `KPIRow` of duty statuses with token tints (tap = filter), `SearchField`, `SegmentedControl`, `FlashList` of `EntityRow` with `MonogramAvatar` and `duty StatusPill`. Row tap opens Detail on the default tab; tapping the hours meta region deep-links to the Hours tab. Invited-but-unaccepted drivers show a neutral Invited pill.

Confirm: tsc passes; subtitles show real load context; the hours deep-link works.

Prompt 8 — Drivers: Invite (Quick Create)

After it runs: compare against [drivers-quick-create.png](#) + the Drivers Quick Create checklist (Section 08).

Read `.planning/mobile-design-system.md` and mockup `drivers-quick-create.png`. Build the Invite Driver sheet on top of the existing driver-invitation flow – consume it as-is, do not modify its API.

FIELD DISCOVERY: from the invitation flow's actual requirements, choose the minimum fields to start the relay (typically name + contact channel). Everything else – license, docs, pay – happens in the driver's own onboarding; the subtitle and caption must say so, because that IS the progress story: the owner's job takes 15 seconds and the system visibly hands off.

BUILD: shared `SheetContainer` + `SheetInputs`, contact field validated as a usable mobile format before the button enables, duplicate contact = inline error naming the existing driver. Success: haptic, dismiss, driver appears with an Invited pill. Resend lives on the driver's Detail page, not here.

Confirm: tsc passes; the sheet reuses shared components; the invitation flow is untouched.

Prompt 9 — Drivers: Detail + Hours tab

After it runs: compare against [drivers-detail-view.png](#), [drivers-detail-edit.png](#), [drivers-hours.png](#) + checklists (Section 08).

Read `.planning/mobile-design-system.md` and mockups `drivers-detail-view.png` / `drivers-detail-edit.png` / `drivers-hours.png`. Build Driver Detail as one component with `isEditing`, mirroring Client Detail, plus a full Hours tab.

FIELD DISCOVERY: list all driver fields; group them per the intelligence rules; expiry-bearing fields (license, docs) get the standard threshold tints and a danger banner on the Docs tab when past due; an address field opens Maps on tap in view mode. Child types found (hours, loads, docs, incidents, pay – whatever exists) become tabs, reusing the shared document row from Trucks. `ParentStrip`: chips for the active Load and assigned Truck.

HOURS TAB: condensed sticky header (small avatar, name, duty pill) with tabs pinned. A `DRIVE TIME TODAY` card: elapsed large and bold against the legal limit, accent remaining figure, progress bar turning warning at 80% and danger at 100% with a violation banner; a second thin bar for the cycle total. Duty status rendered as a read-only `SegmentedControl` reflecting current state (the driver app writes it). A timeline card of the day's duty entries: colored dots by status, description with location/duration, right-aligned times in the tenant timezone, durations as '4h 14m'. Day chevrons to browse previous days; empty day states the last known status. All HOS math in a tested shared util.

EDIT (profile): stacked-label form rule, full parity behavior from the Client Detail prompt.

Confirm: tsc passes; single detail component; bar thresholds fire at 80/100%; document rows are the shared component.

Prompt 10 — Global QA sweep (run last)

After it runs: compare against every page against its mockup + the full Section 09 checklist.

Read `.planning/mobile-design-system.md`. Act as a senior design reviewer, not a builder: find and fix violations across `apps/mobile`, do not redesign anything.

1. Spacing: grep all screens; any margin/padding not in `{8,12,16,20,24}` normalizes to the nearest scale value; screen margin always 20.
 2. Borders: remove every border on cards/rows app-wide.
 3. Accent: per screen, accent may appear only on active tab, primary buttons, tappable text, live emphasis; recolor violations to token colors.
 4. States: every data screen has a content-shaped Skeleton, an EmptyState with a path to the add action, and errors that preserve input; replace any remaining spinner.
 5. Interaction: every Pressable min 48pt, pressed opacity, mapped haptic, accessibilityLabel on icon-only controls; keyboard-avoiding forms.
 6. Parity: per entity, verify one detail component with `isEditing` and identical field order between modes.
 7. Hierarchy: parents only in ParentStrip above tabs; children only as tabs; no inline child sections.
 8. Forms: verify every create/edit surface follows the Form & Field Intelligence rules – exposure tiers, group sizes, field order, defaults, pickers, inline validation.
- Append every fix with file paths to the migration checklist in the design doc.

Confirm: tsc passes; zero borders on cards; zero off-scale spacing; zero spinners on data screens.

Extending the system

Loads, Trips, Invoices, and every future entity get the same four pages. To spec a new entity, answer these five questions and drop the answers into the four-page pattern — nothing else changes:

- What one question does the Overview answer for the owner? (drives the KPI cards)
- What 3–4 fields make the record real? (drives Quick Create)
- Is the entity a person/company (circle avatar, contact actions) or a thing (square chip, no contact row)?
- What are its child records? (drives the Detail tabs)
- Which field states cost the owner money? (drives the warning/danger tints)

Change control

This PDF is generated from the design tokens — it is the contract. Any visual feedback (from users, the founder, or the developer) is first written into the design system, then this document is regenerated, then screens are updated. Rules never change on a single page. That is what keeps every page looking like it came from the same app.

Changelog

v1.2 — Added Form & Field Intelligence rules, the step-by-step build workflow, and one copy-paste Claude Code prompt per page.

v1.1 — Edit mode: labels stack above full-width inputs (view stays label-left). Added Belongs-to parent strip. Locked 5-value spacing scale.

v1.0 — Initial system: tokens, components, four-page pattern, 13 reference screens.

Mockup index

clients-overview.png
clients-quick-create.png
clients-detail-view.png
clients-detail-edit.png
trucks-overview.png
trucks-quick-create.png
trucks-detail-view.png
trucks-detail-edit.png
drivers-overview.png
drivers-quick-create.png
drivers-detail-view.png
drivers-detail-edit.png
drivers-hours.png

All 13 mockups ship alongside this PDF as PNGs (786x1704) for pixel comparison on device.